1- **CLASS TASKS SOLUTIONS**

--- TASK 1, 2, 3 & 4 SOLUTION KEY (PYTHON CODE) ---

```Python

```
import pandas as pd

Task 1: Setup & Basic Grouping
clean_transactions = {
    "transaction_id": ["TXN-701", "TXN-702", "TXN-703", "TXN-704", "TXN-
705", "TXN-706", "TXN-707", "TXN-708", "TXN-709"],
    "category": ["Footwear", "Accessories", "Footwear", "Apparel",
"Accessories", "Apparel", "Footwear", "Apparel", "Accessories"],
    "region": ["North", "South", "East", "North", "West", "South", "West",
"East", "North"],
    "sales_amount": [450.00, 120.00, 890.00, 310.00, 95.00, 420.00, 620.00,
210.00, 150.00],
    "units": [3, 2, 6, 4, 1, 5, 5, 2, 2]
}

df_txns = pd.DataFrame(clean_transactions)

cat_grouped = df_txns.groupby("category")[["sales_amount", "units"]].sum()
print("--- BASIC GROUPBY SUM ---")
print(cat_grouped)

Task 2: Multi-Metric Aggregation
category_summary = df_txns.groupby("category").agg({
    "sales_amount": ["sum", "mean"],
    "units": "sum",
    "transaction_id": "count"
}).rename(columns={"transaction_id": "order_count"})

print("\n--- CATEGORY SUMMARY ---")
print(category_summary)

Task 3: Sorting & Frequency Counts
sorted_summary = category_summary.sort_values(by=("sales_amount", "sum"),
ascending=False)
print("\n--- SORTED SUMMARY ---")
print(sorted_summary)

regional_counts = df_txns["region"].value_counts()
print("\n--- REGIONAL FREQUENCY ---")
print(regional_counts)
```

**--- TASK 4 BUSINESS MEMO SOLUTION KEY ---**

"To: Marcus Thorne, Product Category Manager Re: Category Profitability & Regional Performance Audit
```

An aggregated analysis of transaction data reveals that Footwear generates the highest total sales volume ($1,960.00 across 3 transactions), significantly outperforming Apparel ($940.00) and Accessories ($365.00). Geographically, the North region leads transaction frequency with 3 orders. Recommendation: Focus promotional inventory on high-volume Footwear lines while evaluating marketing campaigns in the North region to capitalize on customer engagement."

2- **HOMEWORK SOLUTION**

```Python

```
Homework Problem 1 & 2 Python Solution
import pandas as pd

omni_sales_data = {
    "order_id": ["ORD-801", "ORD-802", "ORD-803", "ORD-804", "ORD-805", "ORD-806", "ORD-807", "ORD-808", "ORD-809", "ORD-810", "ORD-811", "ORD-812"],
    "channel": ["Online", "In-Store", "Online", "In-Store", "Online", "Online", "In-Store", "In-Store", "Online", "In-Store", "Online", "In-Store"],
    "customer_tier": ["Gold", "Silver", "Platinum", "Bronze", "Gold", "Platinum", "Silver", "Bronze", "Gold", "Platinum", "Silver", "Bronze"],
    "department": ["Electronics", "Apparel", "Electronics", "Home", "Apparel", "Electronics", "Home", "Apparel", "Home", "Electronics", "Apparel", "Home"],
    "net_revenue": [1450.00, 180.00, 2100.00, 95.00, 220.00, 1850.00, 110.00, 140.00, 130.00, 3100.00, 195.00, 85.00],
    "discount_applied": [50.00, 10.00, 100.00, 5.00, 15.00, 75.00, 0.00, 10.00, 10.00, 150.00, 12.00, 5.00]
}

df_omni = pd.DataFrame(omni_sales_data)

Problem 1: Customer Tier Summary
tier_summary = df_omni.groupby("customer_tier").agg({
    "net_revenue": ["sum", "mean"],
    "order_id": "count"
}).rename(columns={"order_id": "order_count"})

tier_sorted = tier_summary.sort_values(by=("net_revenue", "sum"), ascending=False)
print("--- TIER SUMMARY ---")
print(tier_sorted)

Problem 2: Multi-Index Channel & Department Summary
channel_dept_summary = df_omni.groupby(["channel", "department"]).agg({
    "net_revenue": "sum",
    "discount_applied": "mean"
})
print("\n--- CHANNEL & DEPT SUMMARY ---")
print(channel_dept_summary)
```
```

- **Homework Grading Criteria:**
    - Correctly identifies the **Platinum** tier as the most lucrative segment (driven by massive average order values and total revenue).
    - Accurately aggregates multi-index grouping between online and in-store channels across departments.
    - Formulates actionable recommendations focusing acquisition spend on high-value Platinum tier online shoppers.